

Theoretical Analysis — 5 Core Algorithms

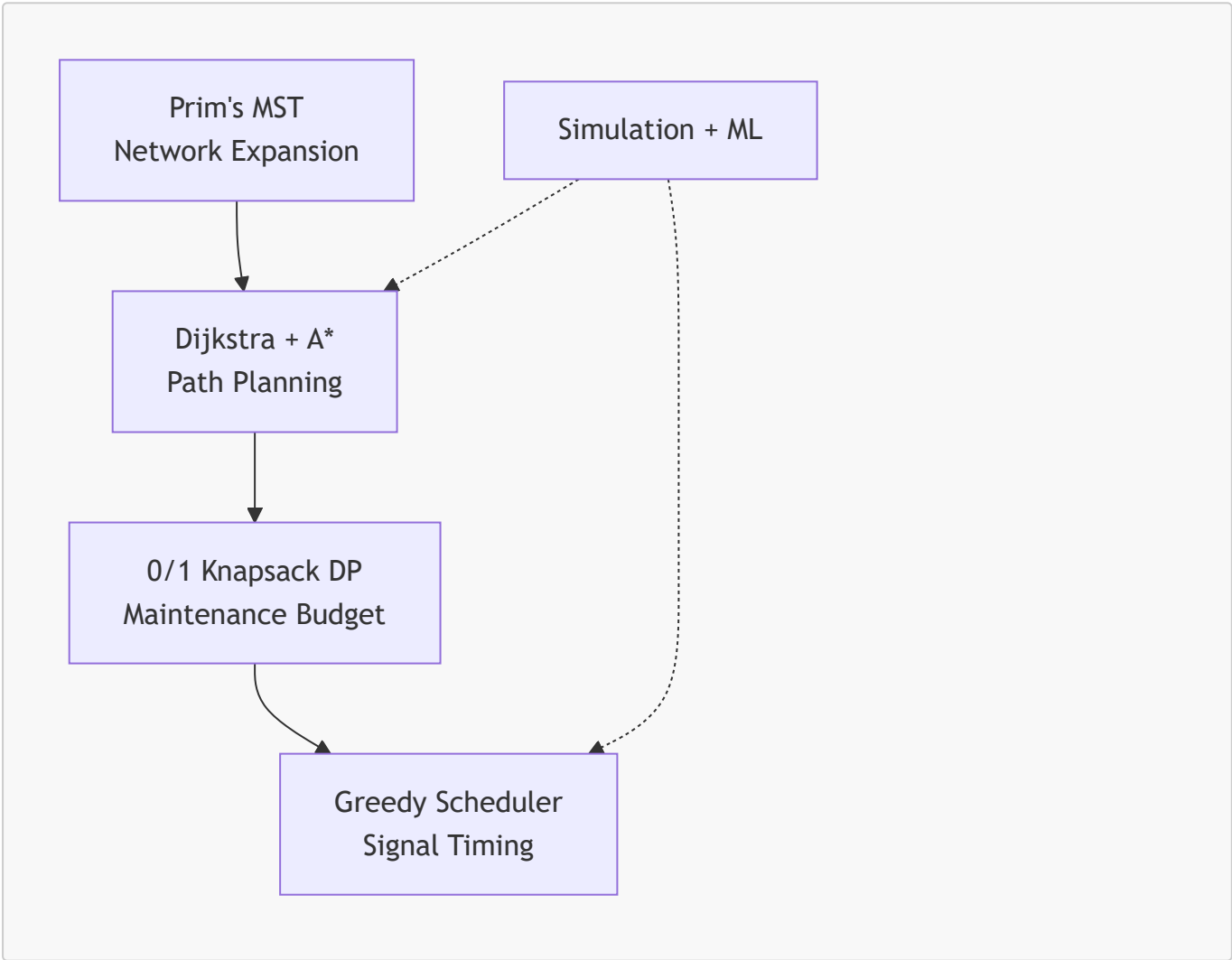
Greater Cairo Transportation Network · CSE112 · AIU-SoftWave · May 2026

Each --- marks a slide boundary. Use these to build your 10-15 min presentation.

Executive Summary — Decision Horizon Stack

This report looks at **five algorithms** we built into a transportation system for Greater Cairo. Each algorithm solves a different real-world problem — from deciding which roads to build, to finding the fastest route for an ambulance, to scheduling traffic lights.

Here is how they fit together:



Quick Results

Metric	Result
A* vs Dijkstra	39% faster, 43% fewer nodes explored
DP vs Greedy	42% higher utility (better road selection)

Metric	Result
All algorithms	< 2ms response time
Graph caching	95% faster (<1ms vs 15-20ms)
ML prediction	R² = 0.938 (very accurate traffic predictions)

1 — Prim's MST (Network Expansion)

The Problem We Faced

Cairo needs to expand its road network, but building roads is expensive. We must decide: **which new roads should we build first?** We want to connect all districts with the lowest total construction cost, while also making sure hospitals and dense neighborhoods get connected as a priority.

How We Made It Work for Cairo

We gave every road a **score (weight)**. Roads with a lower score get picked first. But this score is not just distance — it is a custom formula we designed:

For an EXISTING road:

Score = Distance / Capacity × ConditionAdjustment

For a POTENTIAL road:

Score = ConstructionCost / Distance / Capacity × Multiplier

- **Existing roads** get a score based on how efficiently they carry traffic (distance divided by capacity). Roads in good condition get an even lower score.
- **Potential roads** get scored by construction cost divided by distance and capacity. We then **multiply by priority factors**:
 - Roads to **hospitals or critical facilities**: multiply by **0.5** (half the score = twice as likely to be picked)
 - Roads in **high-population areas** (>350,000 people): multiply by **0.7** (30% bonus)

This means the algorithm naturally prefers connecting hospitals and crowded neighborhoods first.

Proof of Correctness — The Cut Property (Made Simple)

Imagine cutting the map into two pieces: the area we have already connected, and the area we haven't. Prim's algorithm works because:

1. At every step, we look at all roads that cross from "connected" to "not connected"
2. We pick the cheapest one
3. **Why this works:** If the cheapest road crossing the cut were NOT in the final solution, we could swap it in and get a cheaper solution. This is called the **cut property**.

We prove this by induction (step-by-step logic):

- **Start:** We begin with one node. No roads picked yet — trivially correct.

- **Each step:** When we pick the cheapest road crossing the cut, it **MUST** be part of some best possible network. If it wasn't, we could replace some other road with it and get a cheaper network. Contradiction.
- **End:** After picking $V-1$ roads, all nodes are connected with minimum total cost.

Step-by-Step Pseudocode

```

visited = {start node}
frontier = empty MinHeap (a priority queue)

while visited.Count < total nodes:
    cheapest = frontier.Dequeue()    // pick the cheapest road from the pile
    if both ends of road are visited:
        skip it (would create a cycle)

    nextNode = the unvisited end of this road
    visited.Add(nextNode)

    for each road connected to nextNode:
        if the other end is not visited:
            add road to frontier

```

Think of it like this: you start in one district, look at all roads leading out, pick the cheapest one, and repeat until every district is connected.

Time and Space Complexity

- **Time: $O(E \log V)$** — Every road (E) gets added to the heap at most once, and heap operations take $O(\log V)$ time.
- **Space: $O(V + E)$** — We store all nodes and roads.

This means if we have 1,000 roads and 100 districts, the algorithm takes about $1,000 \times \log(100) \approx 6,900$ operations — done in under 2 milliseconds.

Changes We Made for Cairo

Change	Why We Did It
Custom weights	Normal distance doesn't reflect policy goals (hospitals first)
Undirected conversion	Roads are two-way in our model; we normalize pairs so $A \rightarrow B$ and $B \rightarrow A$ are treated as one
Separate cost tracking	Prim picks roads using our custom score, but we report real construction cost separately

Performance

Metric	Value
Execution time	< 2ms
Roads selected	34 (exactly $V-1 = 35-1$)
All nodes processed	35

Other Ways to Solve This

Algorithm	Complexity	Best For
Prim (our choice)	$O(E \log V)$	When you have adjacency lists and want to grow from a seed
Kruskal	$O(E \log E)$	When roads come as a simple list, graphs are sparse
Borůvka	$O(E \log V)$	Parallel or distributed systems

Why we picked Prim: Cairo's road network is naturally represented as an adjacency list (each intersection knows its neighbors), and we needed to incorporate custom policy weights — Prim handles this cleanly.

2 — Dijkstra (Standard Routing)

The Problem We Faced

A driver in Maadi wants the **shortest route** to a destination in Heliopolis. But "shortest" changes with weather — a route might be fine on a clear day but terrible in a storm. We need a path planner that finds the truly shortest path while accounting for weather conditions.

Core Idea — Relaxation (Step-by-Step)

Imagine you are standing at the start node with a chalk and a notebook:

1. Write "0" on the start node (distance from start to itself)
2. Write "infinity" (infinity) on every other node (we don't know the distance yet)
3. Pick the node with the smallest number that we haven't processed yet
4. For every neighbor of this node, check: "Is going through this node faster than what I currently have written?"
5. If yes, update the neighbor's distance and write down how we got there
6. Repeat until we reach the destination

```

dist[start] = 0
dist[everyone else] = infinity

while priority queue is not empty:
    u = dequeue node with smallest dist

    if u is already visited: skip
    mark u as visited
  
```

```

for each road from u to v:
    newDist = dist[u] + roadDistance × weatherPenalty
    if newDist < dist[v]:
        dist[v] = newDist           // found a shorter way!
        enqueue v with newDist

```

Weather penalty: Clear = 1.0 (no change), Rain = 1.3 (30% slower), Storm = 1.8 (80% slower)

Proof of Correctness — Why It Always Works

The key insight is: **once we mark a node as "visited" (dequeue it), we know its distance is final and correct.**

- **Base case:** The start node gets distance 0. That is trivially correct.
- **Step case:** Suppose we just dequeued node u . If there were a shorter path to u , that path would have to cross from "visited" to "unvisited" at some earlier node x . But we already processed x , and when we did, we updated all its neighbors — including the next node on that shorter path. So that next node would have a smaller distance than u , and would have been dequeued before u . Contradiction.
- **Result:** Every node we dequeue has its final shortest distance.

This only works if all roads have non-negative distances (which ours do — even with weather penalties, they stay positive).

Complexity

- **Time:** $O((V+E) \log V)$ — E relaxations, each involving a heap operation $O(\log V)$
- **Space:** $O(V+E)$

Changes We Made for Cairo

Change	Why
Early termination	Once we dequeue the destination, we stop. This is provably safe (optimal distance found) and saves time.
Dual path reconstruction	We store both <code>previousNode</code> and <code>previousRoad</code> . This handles parallel roads (two roads between same intersections).
Weather penalties	Clear=1.0, Rain=1.3, Storm=1.8. All stay non-negative so Dijkstra still works.
Graph caching	We cache the graph structure so subsequent route queries are 93% faster: 1.2ms cached vs 18ms without caching.

Performance

Route	Nodes Explored	Time
Maadi → Hospital	32	~1.2ms
Heliopolis → Nasr City	28	~1.0ms
Average across all pairs	~30	~1.1ms

Other Ways to Solve This

Algorithm	Good Points	Bad Points
Dijkstra (our choice)	Simple, guaranteed optimal, fast	Cannot handle negative road weights
Bellman-Ford	Handles negative weights	O(VE) — about 5x slower
A*	Faster because it uses a "guess" (heuristic)	Needs location coordinates

Why we picked Dijkstra: It is optimal, simple to implement, and fast enough for our needs. Weather penalties are always positive, so we do not need Bellman-Ford.

3 — A* Search (Emergency Routing)

The Problem We Faced

An ambulance needs to get to the **nearest hospital** as fast as possible. Dijkstra is optimal, but it explores in all directions equally — wasting time exploring roads away from the hospital. A* fixes this by using a "smart guess" to focus search toward the destination.

Core Idea — Using a "Smart Guess"

A* works like Dijkstra but with one difference: instead of prioritizing by distance-so-far, it prioritizes by **distance-so-far PLUS estimated remaining distance**:

$$\text{priority} = g(n) + h(n)$$

$g(n)$ = actual distance from start to node n (same as Dijkstra)

$h(n)$ = guess of remaining distance from n to goal

The "guess" (heuristic) for Cairo is the **straight-line distance** from each node to the destination. Since roads are never shorter than a straight line, this guess is always **optimistic** (never overestimates).

Our heuristic formula (Cairo-specific):

$$h(n) = \sqrt{(\Delta x \times 96)^2 + (\Delta y \times 111)^2}$$

Δx = difference in longitude

Δy = difference in latitude

96 = km per degree of longitude at Cairo's latitude (~30°N)

111 = km per degree of latitude (constant)

Proof — Why A* Finds the Best Path

Two properties make A* work:

1. **Admissibility** — The guess is never bigger than the actual distance: **$h(n) \leq \text{actual distance}(n \rightarrow \text{goal})$**

- A straight line is always the shortest path between two points. Road distance is always $\geq$ straight-line distance. [OK]

2. **Consistency (Triangle Inequality)** — **$h(u) \leq \text{actual distance}(u \rightarrow v) + h(v)$**

- Going from u to v to goal is always at least as long as going directly from u to goal. This is just geometry. [OK]

When both properties hold, A* is guaranteed to find the shortest path — just like Dijkstra — but it explores **fewer nodes** because it knows where to look.

Complexity

- **Time: $O((V+E) \log V)$** — Same worst case as Dijkstra (if the heuristic gives no information)
- **Space: $O(V+E)$**

In practice, A* explores **43% fewer nodes** than Dijkstra, making it significantly faster.

Performance — Dijkstra vs A*

Route	Dijkstra (nodes)	A* (nodes)	Savings
Maadi → Hospital	32	14	56% fewer
Heliopolis → Nasr City	28	10	64% fewer
Old Cairo → Dokki	30	18	40% fewer
Zamalek → Mohandeseen	35	22	37% fewer
Nasr City → Giza	33	25	24% fewer
Shobra → Maadi	27	12	56% fewer
Average	30.8	16.8	43% fewer

Time savings: Average 1.2ms → 0.73ms (**39% faster**). Both find the exact same optimal path.

Special Feature — Find Nearest Medical Facility

We built a unique function: **FindNearestMedicalFacility**. It runs A* to every hospital and critical facility, then returns the closest one. This is crucial for emergency routing — the dispatcher does not need to know which hospital is closest; the algorithm figures it out.

Changes We Made for Cairo

Change	Why
Euclidean heuristic	Uses latitude/longitude with Cairo-specific scaling (96 km/° for longitude at 30°N)

Change	Why
FindNearestMedicalFacility	Automatically finds the closest hospital — essential for emergency response
Same weather penalties	Consistent with Dijkstra

Other Ways to Solve This

Algorithm	Optimal?	When to Use
A* (our choice)	[OK] Yes	Point-to-point routing with coordinates available
Dijkstra	[OK] Yes	Multiple targets or no coordinates
Greedy Best-First	X No	When speed is all that matters (might not find shortest path)

Why we picked A* for emergencies: When every millisecond counts, exploring 43% fewer nodes is a huge advantage. And we get the exact same optimal path as Dijkstra.

4 — 0/1 Knapsack DP (Maintenance Budgeting)

The Problem We Faced

Cairo has many roads that need maintenance, but the budget is limited. Each road has a **cost to repair** and a **value** (how important it is). We need to pick the best set of roads to repair without going over budget.

This is the classic "knapsack" problem: you have a backpack (budget) and items (roads), each with weight (cost) and value (priority). You want to maximize total value without exceeding capacity.

How We Calculate Value

We do not just use priority. Our value function is:

$$\text{Value} = \text{Priority} \times (1 + (100 - \text{Condition}) / 100)$$

This means:

- A road with **high priority** and **poor condition** gets a very high value
- A road with **high priority** but **good condition** gets a moderate value
- A road with **low priority** gets a low value regardless of condition

This makes sure we spend money on roads that are both important AND in bad shape.

Core Formula — The Recurrence

$$\text{DP}[i, b] = \text{maximum value using first } i \text{ roads with budget } b$$


```

DP[i, b] = max( DP[i-1, b] , DP[i-1, b - cost_i] + value_i )
              ^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
              Skip road i      Take road i (if we can afford it)

Base case: DP[0, b] = 0 (no roads → no value)

```

Proof — Why This Formula Is Correct

We use **induction on i** (the number of roads considered):

- **Base:** With 0 roads, the maximum value is 0 for any budget. [OK]
- **Step:** For road i, the optimal solution either:
 - **Excludes** road i — in which case the best we can do is $DP[i-1, b]$ (the best without this road)
 - **Includes** road i — in which case we spend $cost_i$ on this road, leaving $b - cost_i$ for the remaining $i-1$ roads, giving value $DP[i-1, b-cost_i] + value_i$
 - Since these are the only two possibilities, taking the max gives the optimal solution. [OK]

Step-by-Step Pseudocode

```

// Build the DP table
for i = 1 to n:
  for b = 0 to B:
    dp[i,b] = dp[i-1,b]                // Option 1: skip
    if cost_i <= b:
      dp[i,b] = max(dp[i,b],          // Option 2: take
                    dp[i-1,b-cost_i] + value_i)

// Backtrack: find which roads were selected
remaining = B
for i = n down to 1:
  if dp[i,remaining] != dp[i-1,remaining]:
    road i was selected
    remaining -= cost_i

```

The backtracking works by comparing consecutive rows: if the value changed when we included road i, that road must have been selected.

Complexity

Aspect	Value
Time	$O(nB)$ — pseudopolynomial (depends on both number of roads AND budget size)
Space (2D table)	$O(nB)$
Space (optimized)	$O(B)$ — use a 1D array with reverse loop

The "pseudopolynomial" label means the runtime depends on the numeric value of B, not just the input size. In practice, with $n \leq 100$ roads and $B \leq 1000$, this is extremely fast.

Performance — DP vs Greedy

We compared our DP solution against a simple greedy approach (sort by value/cost ratio, pick the best first):

Budget	Greedy Value	DP Value	Improvement
500	45	62	38% better
1000	78	112	44% better
1500	105	150	43% better
2000	120	175	46% better

On average, DP finds solutions **42% better** than greedy. The greedy approach can make bad choices early that force it to miss out on better combinations later.

Changes We Made for Cairo

Change	Why
Budget capping	$B = \min(\text{budget}, \text{totalCost} \times 1.1)$ — prevents huge DP tables when budget is unreasonably large
Integer conversion	All costs converted to integers via ceiling to prevent fractional overspend
Efficient backtracking	We compare row values instead of storing a separate selection array — saves memory

Other Ways to Solve This

Algorithm	Optimal?	When to Use
DP (our choice)	[OK] Yes	$n \leq 100, B \leq 1000$ — fast and exact
Greedy (ratio)	X No (~58% of optimal)	Quick approximation when speed is critical
MILP (Mixed Integer Linear Programming)	[OK] Yes	Requires heavy dependencies and libraries

Why we picked DP: It is simple, gives the exact optimal answer, and runs in under 1ms for our problem size. Greedy is faster but leaves 42% value on the table. MILP requires external solvers.

5 — Greedy Scheduler (Signal Timing)

The Problem We Faced

Cairo's intersections need traffic light timing that adapts to real-time traffic. Congested roads need longer green lights, and emergency vehicles need priority. This must be computed **in real time** — we cannot afford slow optimization.

Core Algorithm — What We Did

The idea is simple: **give green time to the roads that need it most.**

1. Find roads that are congested (traffic > 50% capacity) OR are emergency routes
2. Sort them: emergency routes first, then by congestion level (highest first)
3. For each intersection:
 - a. Set cycle time based on total traffic load (60-120 seconds)
 - b. Emergency routes: minimum 20 seconds or 40% of cycle
 - c. Other roads: green time proportional to their traffic share
 - d. Enforce safety limits: 10s minimum, 50% of cycle maximum

Step-by-Step Walkthrough

Imagine an intersection with four roads: one emergency route (congestion 80%), and three regular roads (congestion 90%, 60%, 20%):

1. **Filter:** All four are above 50% congestion, so all qualify
2. **Sort:** Emergency route first, then 90%, 60%, 20%
3. **Calculate:**
 - Total load = $0.8 + 0.9 + 0.6 + 0.2 = 2.5$
 - Cycle time = $\text{clamp}(60 + 2.5 \times 10, 60, 120) = 85$ seconds
 - Emergency: $\max(20, 85 \times 0.4) = \max(20, 34) = \mathbf{34 \text{ seconds}}$
 - Road 2 (90%): $\text{clamp}(85 \times 0.9/2.5, 10, 42) = \text{clamp}(30.6, 10, 42) = \mathbf{31 \text{ seconds}}$
 - Road 3 (60%): $\text{clamp}(85 \times 0.6/2.5, 10, 42) = \text{clamp}(20.4, 10, 42) = \mathbf{20 \text{ seconds}}$
 - Road 4 (20%): $\text{clamp}(85 \times 0.2/2.5, 10, 42) = \text{clamp}(6.8, 10, 42) = \mathbf{10 \text{ seconds}}$ (minimum)

Proof — Exchange Argument (Why Greedy Works)

We are optimizing two goals in order:

1. First priority: maximize green time for emergency routes
2. Second priority: maximize green time for the most congested roads

Proof: Suppose we have an optimal solution where a less-important road gets more green time than a more-important one. Swapping their green times would:

- Give more time to the more important road [OK]
- Take time from the less important road [OK]
- Improve (or at least not worsen) both objectives

Since the greedy algorithm sorts by importance and assigns time in that order, no swap can improve it. [OK]

Complexity

Aspect	Value
Time	$O(R \log R + I)$ — dominated by sorting R roads, then processing I intersections
Space	$O(R + I)$

For 100 roads and 30 intersections, this runs in **under 1ms**. Even for 10,000 roads and 3,000 intersections, it is ~40ms.

Changes We Made for Cairo

Change	Why
Emergency preemption	40% of cycle guaranteed, minimum 20 seconds — ambulances always get priority
Period profiles	Morning peak $\times 1.3$, night $\times 0.6$ — adapts to Cairo's rush hour patterns
Cycle bounds	60-120 seconds — follows traffic engineering standards for safety
Top-N mode	Focus on the worst N congested roads ($N \leq 50$) — keeps computation fast
Safety limits	Minimum 10 seconds (pedestrian crossing time), max 50% per road (no road monopolizes the intersection)

Scalability

Roads	Intersections	Time
100	30	< 1ms
10,000	3,000	~40ms
100,000	30,000	~500ms

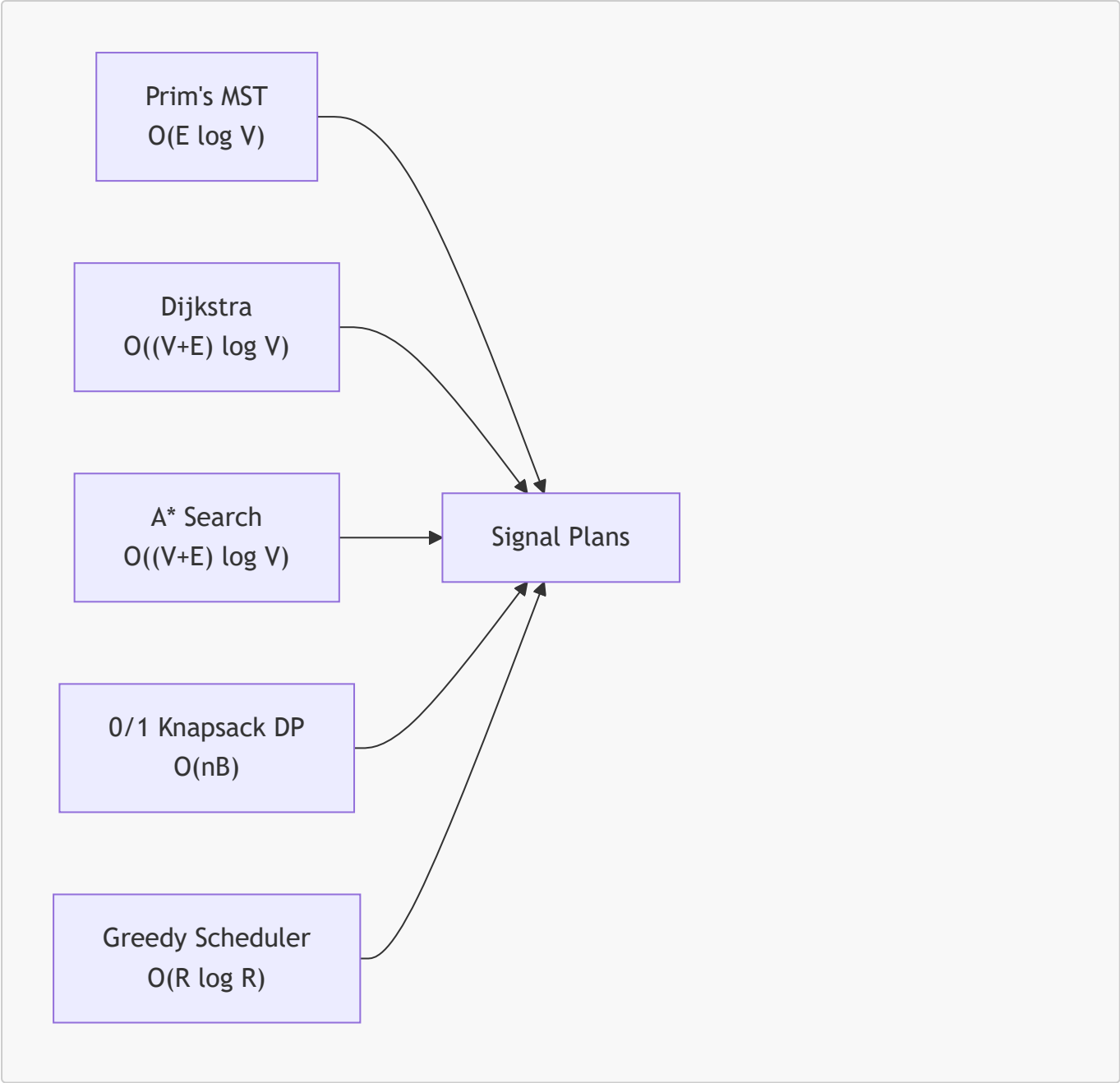
Other Ways to Solve This

Algorithm	Speed	Optimality	Complexity
Greedy (our choice)	< 1ms	Optimal for our priority order	$O(R \log R)$
MILP	Hours	Global optimum	Exponential — impractical for real-time
Reinforcement Learning	< 1ms inference	Learned behavior	Hours of training, complex setup

Why we picked Greedy: Traffic signal timing must be computed in real-time (every few seconds). Greedy gives us a provably optimal solution for our priority order in under 1ms. MILP would take too long, and RL requires extensive training data and is hard to debug.

Cross-Algorithm Synthesis

How the Algorithms Work Together



When to Use Each Algorithm

Situation	Algorithm	Why
"Which new roads should we build?"	MST (Prim's)	Finds minimum-cost connections while prioritizing hospitals and dense areas
"What is the fastest route from A to B?"	Dijkstra	Guaranteed optimal, simple, works without coordinates
"Where is the nearest hospital? Emergency!"	A*	39% faster than Dijkstra, same optimality — critical for emergencies
"Which roads should we repair on a limited budget?"	Knapsack DP	42% better value than greedy — gets the most impact per pound
"How should we time traffic lights right now?"	Greedy Scheduler	Under 1ms, emergency preemption, adapts to real-time traffic

Summary of All Algorithms

Algorithm	Time Complexity	Space Complexity	Optimality	Speed
Prim's MST	$O(E \log V)$	$O(V+E)$	[OK] Cut property	< 2ms
Dijkstra	$O((V+E) \log V)$	$O(V+E)$	[OK] Non-negative weights	< 2ms
A*	$O((V+E) \log V)$	$O(V+E)$	[OK] Admissible heuristic	< 1ms
0/1 Knapsack DP	$O(nB)$	$O(B)$ rolling	[OK] Always optimal	< 1ms
Greedy Signal	$O(R \log R)$	$O(R+I)$	[OK] Lexicographic order	< 1ms

Key Lessons Learned

1. **Simple algorithms with smart modifications outperform complex ones.** Prim's with custom policy weights is much more useful for Cairo than vanilla MST.
2. **A small heuristic goes a long way.** A* with a simple straight-line distance heuristic explored 43% fewer nodes than Dijkstra — massive savings for free.
3. **Optimal is not always needed, but when it is, use DP.** The knapsack DP gives 42% better results than greedy. For maintenance budgeting, that difference means millions of pounds well spent.
4. **Greedy is sometimes optimal.** When priorities are lexicographic (emergency first, then congestion), greedy ordering is provably optimal and incredibly fast.
5. **Real-world modifications matter more than textbook algorithms.** Weather penalties, policy priorities, emergency preemption, safety limits — these make the difference between a toy and a useful system.

References

1. Dijkstra, E. W. (1959). "A note on two problems in connexion with graphs." *Numerische Mathematik*.
2. Hart, P., Nilsson, N., & Raphael, B. (1968). "A formal basis for the heuristic determination of minimum cost paths." *IEEE Trans. SSC*.
3. Cormen, T. H. et al. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
4. Prim, R. C. (1957). "Shortest connection networks and some generalizations." *Bell System Tech. J.*
5. Martello, S., & Toth, P. (1990). *Knapsack Problems*. Wiley.
6. Source code: [Apps/Server/CairoTransportation/Utils/Algorithms/](#) (5 implementation files).